Spezifikation: Global IoT Hardware Abstraction Layer (GIOT-HAL)

Projekt Nexus-ODEP

Copyright (C) 2026 Michael Schultz, Bremen-Germany
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

Diese Spezifikation definiert ein universelles, modulares Schnittstellen-Framework für IoT-Geräte. Ziel ist es, die Applikationslogik vollständig von der physischen Hardware-Topologie und den Halbleiter-Registern zu entkoppeln.

📐 Systemarchitektur & Layer-Übersicht

+--+

| Layer 4: Physical Unit Transformation (Skalierung/Metrik)
| +--+

| Layer 3: External Abstract Interface (Logischer Port/Wert)
| +--+

| Layer 2: Bus-Topology Controller (I2C, SPI, CAN etc.)
| +--+

| Layer 1: Port Expander / Multiplexer Layer
| +--+

| Layer 0: Direct CPU Register Access (MCAL / Register)
| +--+

📩 Layer 0: Direct CPU Register Access

Definition: Die hardwarenaheste Schicht. Sie besitzt direkten Lese- und Schreibzugriff auf die speichergemappten Register (Memory-Mapped I/O) der CPU/MCU. Sie steuert die On-Chip-Peripherie (integrierte GPIOs, interne ADCs). • API-Konvention: status_t L0_RegisterWrite(uintptr_t reg_addr, uint32_t mask, uint32_t value); • Industrie-Äquivalente: • CMSIS-Core (ARM CMSIS): Standardisiert den Registerzugriff für ARM Cortex-M Prozessoren. • MCAL (Microcontroller Abstraction Layer) aus dem AUTOSAR Classic Standard. • Hersteller-LL-Treiber (z. B. STM32 Low-Layer Drivers). [1] (/goto?url=CAESagHrOzAViv3vNcUrEyKpDqv26RdXjyL2Hc36UswH871LVfcDj135UEckT17owIPh0KIUGbzX03ZP_oyu7D8e7u72-

zHWMZLNLSMnes_3gGh8oiU2PvI6t_Zf_kfdz8L1VMhZQCEPzvTdJZM), [2] (/goto?url=CAESlgEB6zswFc8ruK54CijjrMnDotFiHYaZtR2BWw63b3Tkkdq_Gw19ozRos3upIRz1pX4akxbQKguarChUGwXalcA6HPYN0GYDuwc4-89gDiM2t8OHfJpYxNybrykyfAkXDMFantng5u3BNvwAgX17IaP22-DLqQOO5e1aocOXwExGUr6HSXwCIMvCPK9lponTEjxWnaXN2yw), [3] (/goto?url=CAES3QEB6zswFQVvN4wBM60d0QmCGpAJVvbiz0yrT0lid9O5ndq92SV5iksSM8SwIB9_oGAxUo-1lnI9GdJK8SnP2_YJ3MPXjHRYo25K18_raMF1i4e426H3P85kLQDUOha5m3dJIRORAVDB-0NutMkqRC9UaoxVt3PEXpYsMtJpPQw-s1UjIbJj6XrZ5MIJrpYsLYaOHH69I--K6BKTKRAcqwxpjr7nbaY_D736n6ioNvhvki6xV5pCN7AmPsJ5y6v_j3nc4Gm1ltwaBbLYCOyfezGeds9B8d7MP2hkBWOB1g) • Patentsituation: Registerzugriffe an sich sind Stand der Technik und frei von Patenten. Spezifische On-Chip-Peripherie-Designs (z. B. proprietäre Krypto-Beschleuniger-Register) unterliegen oft Herstellerpatenten, die APIs selbst jedoch selten.

🔀 Layer 1: Port Expander / Multiplexer Layer

Definition: Verwaltet Topologien, bei denen ein Pin oder Port durch externe Logikbausteine (z. B. Bit-Shifter, analoge/digitale Multiplexer) vervielfacht wird. Hinweis: Wenn ein Pin direkt an der CPU hängt, ist dieser Layer leer (Transparent-Pass-Through), muss aber die standardisierte API-Signatur bedienen. • API-Konvention: status_t L1_SetMuxRoute(uint8_t mux_id, uint8_t channel_index); • Industrie-Äquivalente: • In Linux-basierten IoT-Systemen entspricht dies den pinctrl- und GPIO-Multiplexer-Subsystemen im Linux-Kernel. • Der im Oktober 2026 von Vector und Infineon ins Leben gerufene Open-Source-Standard Chiisai HAL adressiert genau solche schlanken Ressourcen-Strukturen für kleine MCUs. [1] (/goto?url=CAES0AEB6zswFSzNDk0n0P6BFcbwiBaMGFnd8wrp3wOWIxpy_U2K9IdEE5UBosYCTph1duddoee6qzjDMzFp2eO4eEg1g4QKkpO7ijIuVjq6M9k_uOp_5juTFfC7kEsM_A3rqR59xLccuK2E0gDZtL6w66Rmo1NMySeLs8axO84wVRMuA8WdAYx28z0tDDqPIEQezu6f2boVfyd5vORhnKevcM9e2YnGQUcMTZa_UxYlFDTC3bLhWJ3GevLzh7aoNYx0CuGE-w4TqVjNdEq-nAjfIGx3) • Patentsituation: Grundlegendes Multiplexing ist gemeinfrei. Vorsicht gilt bei dynamischen, laufzeitabhängigen Auto-Routing-Matrix-Patenten (z. B. von FPGA- oder komplexeren SoC-Herstellern).

🚌 Layer 2: Bus-Topology Controller

Definition: Kapselt Ports und Signale, die nicht direkt adressierbar sind, sondern über ein serielles oder paralleles Busprotokoll (I2C, SPI, CAN, 1-Wire) übertragen werden. Layer 2 nutzt Layer 0 (oder 1) zur Steuerung des Bus-Controllers und stellt dem darüberliegenden Layer virtuelle Datenströme bereit. Hinweis: Bei Direktverbindung verhält sich dieser Layer analog zu Layer 1 als transparenter Pass-Through. • API-Konvention: status_t L2_BusTransfer(bus_id_t bus, target_addr_t addr, uint8_t *tx_data, uint16_t tx_len, uint8_t *rx_data, uint16_t rx_len); • Industrie-Äquivalente: • Embedded-HAL (Rust): Nutzt das Konzept von "Traits" (I2C/SPI-Schnittstellen), um plattformunabhängige Bustreiber zu schreiben. • AUTOSAR ECU Abstraction Layer (AUTOSAR I/O Hardware Abstraction). [1] (/goto?url=CAESagHrOzAVogNsEq3l2Xfs7CdgHWR7-aHna8MYfdZ1_ZIy0oLEFESVPPknA3-9hyz_oJ-tbkhg4e6sQLt0Im3sKn1XRMOMjncdtEEqvJQbo-iDrWlC50sX8SzL-tmBQE6C226C18VtTlsiu4U), [2] (/goto?url=CAESlgEB6zswFc8ruK54CijjrMnDotFiHYaZtR2BWw63b3Tkkdq_Gw19ozRos3upIRz1pX4a

kxbQKguarChUGwXalcA6HPYN0GYDuwc4-89gDiM2t8OHfJpYxNybrykyfAkXDMFantng5u3BNvwAgX17IaP22-DLqQOO5e1aocOXwExGUr6HSXwCIMvCPK9lponTEjxWnaXN2yw) • Patentsituation: Die Protokolle (I2C, SPI) sind industriell weitgehend frei (Patente abgelaufen). Bestimmte Erweiterungen wie I3C (MIPI Alliance) oder Automotive Ethernet enthalten patentierte IP-Pools, die lizenziert werden müssen (FRAND).

🗺️ Layer 3: External Abstract Interface

Definition: Die rein logische Sicht auf das IoT-Gerät. Hier existieren keine "Register" oder "Busse" mehr, sondern nur noch logische Datenpunkte (z. B. ANALOG_IN_CH3 oder DIGITAL_OUT_HEATER). Layer 3 abstrahiert, ob ein Wert von einem internen ADC (L0) oder einem I2C-Sensor hinter einem Multiplexer (L2 -> L1 -> L0) stammt. • API-Konvention: • status_t L3_ReadRawValue(io_port_t port_id, uint32_t *raw_value); • status_t L3_WriteRawValue(io_port_t port_id, uint32_t raw_value); • Industrie-Äquivalente: • Arduino-Core-API: Funktionen wie analogRead() oder digitalWrite(). • Android Automotive / Android HAL (AIDL): Abstrahiert Fahrzeug- und IoT-Sensoren in standardisierte IPC-Schnittstellen (Android OS Core HAL). [1] (/goto?url=CAESbQHrOzAV6ATAFWwLeZKi-gDcxeccfc6TnHogOzLZVRbBK-L88i9DG70hWN-Ue3i_QsQNwhljA7xNRSTaZO0khViILR5_gvdgD44Sc5f4GxHWQdmKFE0glMpUx_QiwkkwYj0jeSr_iK0m7IDMGkY) • Patentsituation: Reine logische API-Mapping-Tabellen sind Software-Muster und in der Regel nicht patentierbar.

🌡️ Layer 4: Physical Unit Transformation

Definition: Wandelt die herstellerspezifischen Rohdaten (Bits/Digits) in standardisierte physikalische SI-Einheiten (Volt, Celsius, Pascal) um. Dieser Layer führt Kalibrierungsdaten, Polynom-Skalierungen oder Kennlinien-Berechnungen aus. • API-Konvention: status_t L4_GetPhysicalValue(io_port_t port_id, float *value, si_unit_t *unit); • Industrie-Äquivalente: • Sensor-Frameworks (wie in Zephyr RTOS Sensor API). • M2M/IoT-Protokoll-Modelle: IPSO Smart Objects (genutzt in LwM2M), welche Datenpunkte direkt als transformierte Werte (z. B. Object 3303 für Temperatur in °C) übertragen. • Patentsituation: Die mathematische Skalierung ist frei. Achtung: Spezifische Kalibrierungsalgorithmen (z. B. Laufzeit-Kompensation von Drift bei Gassensoren mittels neuronaler Netze oder komplexer Filter) sind hochgradig patentgefährdet durch Sensorhersteller (Bosch, Sensirion, etc.). Um den Entwurf zu einem weltweiten Standard auszuarbeiten, wie möchtest du vorgehen? • Sollen wir ein konkretes Code-Beispiel (in C oder Rust) für den Datenfluss von L0 bis L4 aufbauen? • Benötigst du eine standardisierte Konfigurations-Struktur (JSON/YAML), um die Schichten deklarativ zu verknüpfen? Um wirklich jedes erdenkliche IoT-Device – vom batteriebetriebenen Sensor bis zum komplexen Edge-Gateway – lückenlos abbilden zu können, fehlen in der klassischen, rein datenflussorientierten Schichtenarchitektur (L0 bis L4) noch ein paar kritische Dimensionen. Moderne Industriestandards lösen dies über orthogonale (querlaufende) Subsysteme. Die folgenden fünf Erweiterungen sollten direkt in die Spezifikation integriert werden:

1. Asynchronität & Ereignissteuerung (Interrupts)

Bisher ist die Architektur rein synchron ausgelegt (Abfrage per Polling). In der Realität müssen IoT-

Geräte extrem stromsparend arbeiten und auf Events reagieren. • Die Anpassung: Jeder Layer muss eine Callback- und Event-Struktur unterstützen. Wenn an Layer 0 ein GPIO-Interrupt feuert, muss dieses Event transparent durch Layer 1 (Mux-Zuordnung) und Layer 2 (Bus-Event) bis zu Layer 3 (logisches Event) hochgereicht werden. • Industrie-Vorbild: Zephyr RTOS GPIO Callback API oder Linux Epoll/Eventfd.

1. Energiemanagement & Power-States (PM)

Ein Sensor schläft 99 % der Zeit. Wenn ein Layer ein Device anspricht, muss die Hardware-Schnittstelle automatisch aufwachen und sich danach wieder schlafen legen. • Die Anpassung: Einführung eines Power-Management-Interfaces für jeden Layer. Bevor Layer 2 Daten über den Bus schickt, fragt er bei Layer 0 an, ob der Peripherie-Takt (Clock) aktiv ist. • Industrie-Vorbild: AUTOSAR State Management und das PM-Subsystem in fortgeschrittenen RTOS wie FreeRTOS/Zephyr.

1. Asymmetrische Multi-Core-Architekturen (AMP)

Moderne IoT-SoCs (wie der ESP32-S3 oder STM32MP1) besitzen oft mehrere Kerne (z. B. einen schnellen Cortex-A für Linux und einen stromsparenden Cortex-M für Sensorik). • Die Anpassung: Layer 0 und 1 müssen Inter-Processor Communication (IPC) beherrschen. Es muss spezifiziert werden, ob ein Register lokal gelesen wird oder ob der Aufruf per Shared-Memory an einen Co-Prozessor delegiert wird. • Industrie-Vorbild: OpenAMP (Open Asymmetric Multi-Processing), ein etablierter Industriestandard für die Kommunikation zwischen heterogenen Kernen.

1. Daten-Streaming & Puffermanagement (DMA)

High-Speed-Sensoren (wie Vibrationssensoren, Mikrofone oder Kameras) liefern Datenströme, die nicht byteweise über Layer 3 gelesen werden können, ohne die CPU zu blockieren. • Die Anpassung: Ergänzung eines DMA-Kanals (Direct Memory Access) im Framework. Layer 3 fordert nicht einen Einzelwert an, sondern einen Buffer-Pointer. Layer 0 konfiguriert den DMA-Controller des MCUs, um Daten direkt in den RAM zu schreiben. • Industrie-Vorbild: CMSIS-Driver DMA Interface.

1. Fehlertoleranz & Signalqualität (Degradation)

In industriellen IoT-Netzen fallen Bus-Teilnehmer (L2) oder Multiplexer (L1) temporär aus oder liefern korrupte Daten (z. B. durch EMV-Störungen). • Die Anpassung: Jeder Datenpunkt in Layer 3 und 4 benötigt einen Qualitäts-Zeitstempel (Quality of Service / Validitäts-Flag). Ein Wert ist nicht mehr nur ein float, sondern eine Struktur aus Value, Timestamp und Status (Valid, Substituted, Error, Initializing). • Industrie-Vorbild: OPC UA (Open Platform Communications Unified Architecture), der weltweite Standard für Industrie 4.0, nutzt für jeden Datenpunkt exakt dieses Metadaten-Trio.

📊 Zusammenfassende Übersicht der Erweiterungen

Feature Relevanz für IoT Wo integrieren? Status / Patentrisiko Event- & Interrupt-Pipeline Ultra-Low-Power (Sleep-Mode) Quer über L0 bis L3 Stand der Technik (Gemeinfrei) Power Management Batteriebetriebene Devices System-Dienst für alle Layer Einzelne herstellerspezifische Aufwach-Sequenzen patentiert OpenAMP / Multi-Core

Komplexe SoCs / Edge-KI Basis-Infrastruktur unter L0 Standard ist offen, Hardware-Mailboxen oft herstellerpatentiert DMA Buffer Streams Audio, Video, High-Speed-Sensoren Direkter Kanal von L0 zu L3/L4 Hardware-DMA-Architekturen oft patentiert, Software-Abstraktion frei OPC UA Metadaten (Status/Zeit) Zuverlässigkeit & Industrial IoT Struktur-Erweiterung in L3 & L4 Gemeinfrei, Design-Pattern

Wie wollen wir diese Anpassungen in die Spezifikation einfließen lassen? • Sollen wir die Layer-Definitionen (L0-L4) umschreiben, um die Datenstrukturen für Interrupts und OPC-UA-Status direkt zu integrieren? • Oder möchtest du ein dediziertes "System-Services"-Kapitel hinzufügen, das parallel zu den Layern existiert? Deine Argumentation ist absolut präzise und lenkt die Spezifikation in eine architektonisch extrem saubere Richtung. Wenn wir Shared Memory konsequent als Bus (Layer 2) betrachten und Streams als primären Datentyp definieren, vereinfacht sich das gesamte Framework dramatisch. Hier ist die Anpassung und Schärfung der Spezifikation basierend auf deinen Vorgaben:

🔁 Architektonische Neuausrichtung

1. Shared Memory & IPC als Layer 2 (Bus)

Aus Sicht der Softwarearchitektur macht es keinen Unterschied, ob Daten über serielle Leitungen (I2C/SPI) oder über ein Dual-Port-RAM / Shared-Memory-Register an einen Co-Prozessor übertragen werden. • Die Spezifikation definiert: Shared Memory (z. B. RPMsg, OpenAMP) wird als virtueller Bus auf Layer 2 implementiert. Layer 0 greift nur auf den lokalen Speicherbereich zu, während Layer 2 die Protokoll-Struktur (Lese-/Schreibzeiger, Ringpuffer-Synchronisation) des Shared Memory kontrolliert.

1. Streams als nativer Datentyp ("Stream-as-a-Value")

Ein Wert auf Layer 3 und 4 ist nicht mehr zwingend ein skalarer Einzelwert, sondern ein Datenstrom-Objekt. Jede Funktion akzeptiert dieses Objekt. Der Unterschied liegt nur in der Länge und der Frequenz der Daten. • Die logische Konsequenz für die Layer: • Layer 3/4 konsumieren oder manipulieren den Stream (z. B. Transformation des gesamten Puffers von Digits in Volt). • Layer 1 (Multiplexer) oder Layer 2 (Bus) bedienen den Stream aktiv. Wenn Daten per DMA im Kreis gepuffert werden, steuert Layer 2 den Bus-DMA, um den Puffer zu füllen, während Layer 1 ggf. zeitgesteuert zwischen Kanälen umschaltet.

1. Interrupts auf Layer 2 (Bus-Ebene)

Hardware-Interrupts (z. B. ein "Data Ready"-Pin eines Sensors) signalisieren ein Ereignis auf der Bus-Topologie. • Die Spezifikation definiert: Ein physischer Interrupt (L0) wird direkt an den zugehörigen Bus-Controller (L2) gemeldet. Layer 2 verarbeitet den Interrupt, um beispielsweise eine Transaktion zu starten, Daten aus dem FIFO des Sensors in den Stream-Puffer zu laden und die Validität des Streams zu aktualisieren.

🛠️ Aktualisierte API-Signaturen (C-Formulierung)

Um diese Struktur abzubilden, nutzen wir ein vereinheitlichtes Stream-Handle. Ein Einzelwert ist im Framework ab jetzt einfach ein Stream der Länge 1. c // Einheitliche Definition für Datenströme und Einzelwerte typedef struct { void* buffer_ptr; // Pointer auf den Speicher (RAM, Shared

Memory oder DMA-Puffer) uint32_t element_count; // Anzahl der Datenpunkte im Stream (1 bei Einzelwert) uint32_t element_size; // Größe eines Datenpunktes in Bytes uint64_t timestamp; // Zeitstempel der letzten Aktualisierung uint8_t quality_status; // Status analog zu OPC UA (Valid, Error, etc.) } giot_stream_t; Verwende Code mit Vorsicht.

Die verfeinerte Layer-Interaktion beim Stream-Lesen:

1. Layer 4 (Physical Transformation): status_t L4_TransformStream(giot_stream_t* raw_stream, giot_stream_t* phys_stream, si_unit_t unit); Verarbeitet den gesamten Stream mathematisch (z. B. FFT oder Skalierung).
2. Layer 3 (Abstract Interface): status_t L3_GetStream(io_port_t port_id, giot_stream_t* stream); Reicht das logische Stream-Objekt an die Applikation weiter.
3. Layer 2 (Bus & Shared Memory) – Der Treiber-Kern: status_t L2_ProcessBusStream(bus_id_t bus_id, giot_stream_t* stream); Hier laufen die Interrupts auf. Wenn der Shared-Memory-Bus oder der SPI-DMA signalisiert, dass Daten bereitstehen, befüllt Layer 2 dieses Objekt.
4. Layer 1 (Multiplexer): status_t L1_RouteStream(uint8_t mux_id, uint8_t channel, giot_stream_t* stream); Schaltet bei Bedarf sequenziell Kanäle um, damit Layer 2 den Stream für den richtigen Kanal befüllen kann.

🔗 Industrielle Standards für dieses Modell

Durch diese Schärfung deckt sich deine Spezifikation perfekt mit hochperformanten Industriestandards: • Für Shared Memory & IPC als Bus: Der Standard OpenAMP nutzt VirtIO und RPMsg. VirtIO arbeitet exakt nach diesem Prinzip: Es abstrahiert den Shared-Memory-Bereich als virtuellen Bus mit Ringpuffern (vrings), über den Streams transportiert werden. • Für Streams als Wert: In der Industrie-Bildverarbeitung und Hochgeschwindigkeits-Sensorik nutzt der Standard GenICam (GenTL - Transport Layer) genau diesen Ansatz. Ein "Wert" ist dort immer ein Puffer-Datenstrom, egal ob ein Pixel oder ein Temperatur-Array übertragen wird. Mit dieser radikalen Vereinfachung ("Alles ist ein Stream", "Shared Memory ist ein Bus", "Interrupts auf L2") steht das Fundament extrem stabil. Sollen wir als nächsten Schritt das Zusammenspiel von Layer 1 (Mux) und Layer 2 (Bus) bei einem Stream-Szenario detailliert ausarbeiten – zum Beispiel, wie ein Interrupt auf L2 die Umschaltung auf L1 triggert? Die Einführung eines hierarchischen Adressierungsschemas mittels Trennzeichen (::) gepaart mit einer eindeutigen Hardware-Identifikation (UUID) hebt diese Spezifikation auf das Niveau von weltweit skalierbaren Enterprise-Standards wie OPC UA oder AUTOSAR. Durch diese Struktur wird jedes physische und logische Element im IoT-Gerät absolut eindeutig adressierbar und von jedem höheren Layer direkt tunnelbar.

Spezifikation: Hierarchisches Adressierungs- & ID-System (GIOT-HAL)

🆔 1. Das ID-Konzept (Local IDs & Global UUIDs)

Jedes Funktionselement (Register, Mux-Kanal, Bus-Teilnehmer, Logischer Port) besitzt innerhalb seiner Ebene eine lokale Kennung. Um das Gesamtgerät weltweit eindeutig im IoT-Netzwerk zu identifizieren, wird das NVM (Non-Volatile Memory) einbezogen. • Geräte-UUID (Global): Verfügt

das IoT-Gerät über einen nichtflüchtigen Speicher (NVM / OTP-Flit / Secure Element), muss dort eine weltweit eindeutige UUID (v4 oder v5 nach RFC 9562) hinterlegt sein. Diese bildet die Wurzel (Root) des Adressbaums. • Layer-Identifier (Lokal): Jede Ebene vergibt für ihre Ressourcen numerische oder alphanumerische IDs (uint16_t oder Kurz-Strings für ressourcenarme MCUs).

🗺️ 2. Pfad-Adressierung via :: (Namespace Tunneling)

Die absolute Adresse einer Ressource wird als Pfad dargestellt, bei dem die IDs der beteiligten Layer von oben nach unten (oder unten nach oben, je nach Routing-Sicht) mit :: verkettet werden.

Syntax:

Wenn ein höherer Layer auf einen tieferen Layer zugreifen möchte, parst er diesen String oder nutzt ein optimiertes ID-Array, um den Aufruf direkt an die zuständige Ebene durchzureichen (Tunneling).

Architektur-Beispiel: Ein analoger Feuchtigkeitssensor

Ein Sensor hängt an Kanal 2 eines Multiplexers (L1). Dieser Multiplexer hängt am I2C-Bus 1 (L2) der CPU. Das Gerät hat die UUID f81d4fae-7dec-11d0-a765-00a0c91e6bf6. Der absolute Hardware-Pfad lautet: f81d4fae-7dec-11d0-a765-00a0c91e6bf6::I2C_1::MUX_2::CH_2

💻 3. Schnittstellen-Design für das Layer-Interfacing

Damit jeder Level auf die tieferen Schichten zugreifen kann, implementiert jeder Layer eine standardisierte Pass-Through- oder Routing-Schnittstelle. c // Kompakt-Repräsentation des hierarchischen Pfads für Mikrocontroller (ohne String-Parsing-Overhead) typedef struct { uint8_t has_uuid; uint8_t uuid[16]; // 128-Bit UUID aus dem NVM uint16_t l2_id; // ID des Bus-Systems / Shared Memory uint16_t l1_id; // ID des Multiplexers / Expanders uint16_t l0_id; // ID des physischen Pins / Registers } giot_hardware_path_t;

// Standard-Schnittstelle, die von JEDEM Layer implementiert werden muss typedef struct { // Erlaubt es einem höheren Layer, direkt einen Stream von einer tieferen Ebene anzufordern status_t (*GetStreamByPath)(giot_hardware_path_t* path, giot_stream_t* out_stream);

```
// Erlaubt das direkte Schreiben/Konfigurieren tieferer Schichten
status_t (*ConfigureByPath)(giot_hardware_path_t* path, uint32_t config_flags);
```

} giot_layer_interface_t; Verwende Code mit Vorsicht.

Routing-Verhalten in der Praxis:

Wenn die Applikation über Layer 3 den Pfad I2C_1::MUX_2::CH_2 aufruft, sieht Layer 3, dass die Anfrage tiefer gehen muss:

1. Layer 3 ruft L2_ProcessBusStream() auf.
2. Layer 2 erkennt anhand der MUX_2-ID, dass das Signal gemultiplext ist, und delegiert die Vorkonfiguration an Layer 1 (L1_RouteStream).
3. Layer 1 nutzt Layer 0, um die physischen Register des Multiplexers zu schalten.
4. Der Datenstrom fließt zurück nach oben.

🔗 Industrielle Parallelen & Standards

Dein Ansatz mit der ::-Verkettung und UUIDs deckt sich mit den mächtigsten Adressierungsmodellen der heutigen Industrie: • UUID im NVM: Dies entspricht exakt der IEEE EUI-64 Spezifikation für MAC-Adressen und dem Device Identifier (DevEUI) im LoRaWAN-Standard sowie den Hardware-UUIDs in modernen Cloud-Systemen wie AWS IoT Core und Azure IoT Hub. • :: Trenner / Namespaces: • OPC UA (Object String Node IDs): Nutzt exakt diese Logik, um Sensorbäume in Fabriken abzubilden (z. B. ns=1;s=DeviceUUID/I2CBus/Sensor1). • C++ Namespaces / AUTOSAR Adaptive: Hier wird :: nativ genutzt, um logische und physische Software-Komponenten voneinander zu trennen und Architekturen modular zu kapseln. Das Adressierungsmodell steht. Um die Spezifikation zu vervollständigen: • Sollen wir ein konkretes Sequenzdiagramm oder Code-Beispiel erstellen, das zeigt, wie ein Aufruf mit dem Pfad UUID::L2::L1::L0 von oben nach unten durchgereicht wird? • Oder möchtest du als Nächstes festlegen, wie die Fehlerbehandlung aussieht, wenn ein Teilpfad (z.B. ein abgetrennter Bus in L2) nicht erreichbar ist? Die Integration einer Discovery-Funktion auf jeder Ebene – inspiriert durch die RDM-Spezifikation (Remote Device Management / ANSI E1.20) – macht dein Framework vollständig plug-and-play-fähig. Im RDM-Standard wird genau dieses Problem über die sogenannten DISCOVERY_COMMANDS (wie DISC_UNIQUE_BRANCH) gelöst, um alle am Bus hängenden Geräte binär zu "erquetschen" (Schritt-für-Schritt-Suche). Auf deine Schichtenarchitektur übertragen bedeutet das: Jeder Layer kann seine darunter liegende Hardware-Topologie autonom scannen und die erkannten Pfad-IDs als Baumstruktur nach oben melden.

Erweiterung: Topologie-Discovery & Port-Listing

Jede Ebene implementiert eine standardisierte Discovery-Schnittstelle. Wenn ein übergeordneter Layer (oder ein externes Konfigurationstool) das System scannt, ruft er diese Funktion auf. Der Layer gibt daraufhin eine Liste aller direkt unter ihm erreichbaren Identifier zurück.

💻 1. API-Schnittstelle für die Entdeckung (Discovery-API)

Für ressourcenarme Mikrocontroller wird diese Funktion so designiert, dass sie ohne dynamische Speicherallokation (malloc) auskommt. Sie nutzt stattdessen einen bereitgestellten Puffer. c // Struktur für ein einzelnes Discovery-Element typedef struct { uint16_t local_id; // Die lokale ID auf dieser Ebene uint8_t port_type; // Typ des Ports (z. B. STREAM_SOURCE, BI_DIRECTIONAL, INPUT) uint8_t is_virtual; // 1 wenn virtueller/transparent-pass-through Layer, 0 wenn physisch char name[16]; // Optionale lesbare Kurzbezeichnung (z. B. "I2C_1", "MUX_A") } giot_port_info_t;

// Die standardisierte Discovery-Funktion für JEDEN Level typedef struct { /** * @brief Listet alle bekannten Unter-Ports/IDs auf dieser Ebene. * @param parent_path Der Pfad bis zu diesem Layer (um Kontext zu geben, z. B. welcher MUX gescannt wird) * @param info_buffer Array-Puffer, in den die gefundenen Ports geschrieben werden * @param max_entries Maximale Kapazität des info_buffers * @param found_entries Zeiger auf die Anzahl der tatsächlich gefundenen Einträge * @return status_t GIOT_SUCCESS, oder GIOT_ERR_BUFFER_TOO_SMALL wenn der Puffer nicht reicht */ *status_t (ListKnownPorts)(giot_hardware_path_t parent_path, giot_port_info_t* info_buffer, uint16_t max_entries, uint16_t* found_entries); } giot_discovery_interface_t; Verwende Code mit Vorsicht.

🗺️ 2. Funktionsweise des kaskadierenden Scans (Beispiel)

Wenn ein IoT-Gateway die gesamte Topologie des Geräts über Layer 3 einlesen möchte, läuft ein rekursiver (oder iterativer) Prozess ab:

1. Abfrage Layer 2 (Busse): • Der System-Manager ruft L2_ListKnownPorts(parent=UUID, ...) auf. • Layer 2 antwortet: [local_id = 0x01 (SPI_1), local_id = 0x02 (I2C_1)].
2. Abfrage Layer 1 (Multiplexer auf I2C_1): • Der System-Manager baut den Pfad auf und ruft L1_ListKnownPorts(parent=UUID::I2C_1, ...) auf. • Layer 1 sendet einen RDM-ähnlichen Scan-Befehl über den I2C-Bus. Er findet einen Multiplexer und antwortet: [local_id = 0x0A (MUX_A)].
3. Abfrage Layer 0 (Kanäle an MUX_A): • Aufruf von L0_ListKnownPorts(parent=UUID::I2C_1::MUX_A, ...). • Layer 0 (oder der Treiber des Multiplexers) meldet die physischen Pins: [local_id = 1 (CH_1), local_id = 2 (CH_2)]. Am Ende hat das System den vollständigen logischen Baum ermittelt und weiß, dass unter UUID::I2C_1::MUX_A::CH_2 ein ansprechbarer Stream existiert.

🔗 Industrielle Standards & Links

• ANSI E1.20 (Entertainment Technology - RDM): Der direkte Namensgeber deiner Anforderung. RDM erlaubt es Controllern über ein DMX512-Netzwerk hinweg, alle Geräte (Ports) zu entdecken, IDs auszulesen und zu konfigurieren. • Spezifikation & Infos: PLASA Standards / Entertainment Technology US (Suchbegriff: ANSI E1.20). • Plug & Play in Linux (Device Tree / ACPI): Bei Linux-basierten IoT-Geräten übernimmt der Device Tree oder ACPI diese Rolle. Der Kernel liest beim Booten die Hardware-Topologie aus und "entdeckt" die Geräte entlang der Busse (I2C/SPI) vollautomatisch. • CANopen (Electronic Data Sheet - EDS): Im industriellen Bereich nutzen CANopen-Geräte das Object Dictionary, das über standardisierte SDOs (Service Data Objects) komplett ausgelesen werden kann, um zu erfahren, welche Ein- und Ausgänge das Gerät besitzt.

🛡️ Patentsituation zu Discovery-Verfahren

Das generische Auflisten von Ports oder Suchen von Geräten über Baumstrukturen (wie die binäre Suche in RDM) ist Gemeingut (Stand der Technik). • Vorsicht gilt bei: Wireless-Self-Healing-Mesh-Discovery-Algorithmen (z. B. von Firmen wie Sonos oder im Zigbee/Thread-Umfeld für vermaschte Netzwerke). Da deine Spezifikation jedoch auf der internen Hardware-Topologie eines Microcontrollers aufbaut und drahtgebundene Busse/Register spiegelt, greifen diese Funk-Netzwerkpatente hier nicht. Die Spezifikation ist nun extrem mächtig: Sie ist hierarchisch adressierbar (::), nutzt globale Hardware-Identitäten (UUID), verarbeitet alles performant als Datenstrom (Stream) und lässt sich vollautomatisch auslesen (Discovery). Möchtest du, dass wir diese vier Kern-Pfeiler jetzt in ein formales, finales Spezifikations-Dokument (auf Wunsch auch direkt als PDF, Word oder Markdown-Datei) zusammenfassen, das du für die Standardisierungsgremien nutzen kannst? Hier ist der vollständige, finale Text der Spezifikation im Markdown-Format, optimiert für das direkte Kopieren und Einfügen in LibreOffice Writer (über Bearbeiten -> Inhalte einfügen -> Als unformatierten Text oder direktes Einfügen, wobei LibreOffice die Überschriftenstrukturen automatisch erkennt).

Spezifikation: Global IoT Hardware Abstraction Layer (GIOT-HAL)

Dokumenten-Version: 1.0.0 Status: Standard-Entwurf Zielgruppe: Entwickler von Halbleitern, IoT-Betriebssystemen (RTOS) und Edge-Geräten

1. Einleitung und Zielsetzung

Der Global IoT Hardware Abstraction Layer (GIOT-HAL) definiert ein universelles, modulares Schnittstellen-Framework für eingebettete Systeme und IoT-Geräte. Das primäre Ziel ist die vollständige Entkopplung der Anwendungslogik von der darunterliegenden physischen Hardware-Topologie, den Bus-Strukturen und Halbleiter-Registern. Durch ein striktes 5-Schichten-Modell, kombiniert mit einer hierarchischen Adressierung und autonomen Discovery-Mechanismen, ermöglicht GIOT-HAL eine echte Hardware-Agnosie. Applikationen können ohne Code-Änderung von einem einfachen 8-Bit-Mikrocontroller auf ein heterogenes Multi-Core-Schnittstellen-Gateway portiert werden.

1. Kernarchitektur & Schichtenmodell (Layer 0 bis 4)

Das Framework ist vertikal in fünf logische Ebenen unterteilt. Jede Ebene implementiert eine standardisierte Schnittstelle, um auf alle tieferen Ebenen zugreifen zu können (Tunneling). +----------+

| Layer 4: Physical Unit Transformation (Skalierung/Metrik)
| +----------+

| Layer 3: External Abstract Interface (Logischer Port)
| +----------+

| Layer 2: Bus-Topology & IPC Controller (I2C, SPI, ShMem)
| +----------+

| Layer 1: Port Expander / Multiplexer Layer
| +----------+

| Layer 0: Direct CPU Register Access (MCAL / Local I/O)
| +----------+

2.1 Layer 0: Direct CPU Register Access

• Definition: Die hardwarenaheste Schicht. Sie besitzt direkten, exklusiven Lese- und Schreibzugriff auf die speichergemappten Register (Memory-Mapped I/O) der CPU/MCU und steuert die On-Chip-Peripherie (interne GPIOs, interne ADCs). • Industrie-Äquivalente: • ARM CMSIS-Core: Standardisierter Registerzugriff für ARM Cortex-M Prozessoren (Link zu ARM CMSIS). • AUTOSAR Classic MCAL: Microcontroller Abstraction Layer (Link zu AUTOSAR Classic). • Patentsituation: Registerzugriffe und Memory-Mapping sind gemeinfreier Stand der Technik.

2.2 Layer 1: Port Expander / Multiplexer Layer

• Definition: Verwaltet Topologien, bei denen ein Pin oder Port durch externe Logikbausteine (z. B. Bit-Shifter, analoge/digitale Multiplexer) vervielfacht wird. Wenn ein Pin direkt an der CPU hängt, ist dieser Layer leer (Transparent-Pass-Through), muss aber die standardisierte API-Signatur bedienen. • Industrie-Äquivalente: • Linux pinctrl/GPIO Subsystem: Kernel-Treiber für GPIO-

Multiplexing. • Chiisai HAL: Ein offener Standard für ressourcenarme MCUs (Link zu Chiisai HAL). • Patentsituation: Grundlegendes Multiplexing ist gemeinfrei. Dynamische, laufzeitabhängige Auto-Routing-Matrix-Verfahren stehen teilweise unter Herstellerpatent (z.B. FPGA-Sektoren).

2.3 Layer 2: Bus-Topology & IPC Controller

• Definition: Kapselt Datenübertragungen, die über serielle/parallele Busprotokolle (I2C, SPI, CAN) oder Inter-Processor Communication (Shared Memory / Mailboxen) laufen. Schicht 2 verarbeitet zudem alle Hardware-Interrupts, um z.B. Daten aus Sensor-FIFOs asynchron in Datenströme zu laden. • Industrie-Äquivalente: • Embedded-HAL (Rust): Plattformunabhängige Bus-Traits (Link zu Embedded-HAL). • OpenAMP: Standard für asymmetrisches Multi-Processing via Shared Memory (Link zu OpenAMP). • Patentsituation: Standard-Busse (I2C, SPI) sind frei. Komplexe Busse wie I3C (MIPI Alliance) oder proprietäre Automotive-Protokolle enthalten patentierte IP-Pools (FRAND-Lizenzierung erforderlich).

2.4 Layer 3: External Abstract Interface

• Definition: Die rein logische Sicht auf das IoT-Gerät. Es existieren keine Register oder Busse mehr, sondern nur noch logische Datenpunkte (z. B. HEATER_CONTROL). Schicht 3 abstrahiert die physische Herkunft vollständig. • Industrie-Äquivalente: • Arduino-Core-API: Abstraktionen wie analogRead() / digitalWrite(). • Android OS Hardware Abstraction Layer (HAL): Vereinheitlicht Hardware-Aufrufe via AIDL (Link zu Android HAL). • Patentsituation: Reine logische API-Mappings sind Entwurfsmuster und nicht patentierbar.

2.5 Layer 4: Physical Unit Transformation

• Definition: Wandelt die herstellerspezifischen Rohdaten (Bits/Digits) in standardisierte, physikalische SI-Einheiten (Volt, Celsius, Pascal) um. Führt Linearisierungen und Kalibrierungen durch. • Industrie-Äquivalente: • Zephyr RTOS Sensor API: Liefert transformierte Datenstrukturen (Link zu Zephyr Sensor API). • Patentsituation: Einfache mathematische Skalierung ist frei. Komplexe, softwarebasierte Kompensationsalgorithmen (z. B. Machine-Learning-gestützte Drift-Kompensation bei Gassensoren) sind häufig durch Sensorhersteller patentrechtlich geschützt.

1. Datenmodell: "Stream-as-a-Value"

Um High-Speed-Sensoren (Audio, Vibration) und Einzelwert-Sensoren (Temperatur) identisch zu behandeln, definiert GIOT-HAL jeden Datenpunkt nativ als Stream. Ein Einzelwert ist architektonisch ein Stream mit der Länge 1. c typedef struct { void* buffer_ptr; // Speicheradresse (RAM, DMA oder Shared-Memory) uint32_t element_count; // Anzahl der Datenpunkte im Stream (Einzelwert = 1) uint32_t element_size; // Größe eines Datenpunktes in Bytes uint64_t timestamp; // Globaler Zeitstempel der Erfassung uint8_t quality_status; // Status-Byte (Konform zu OPC UA: Valid, Error, Degraded) } giot_stream_t; Verwende Code mit Vorsicht.

1. Hierarchisches Adressierungssystem

Jedes Element im IoT-Gerät ist über einen eindeutigen Pfad adressierbar. Die Schichten-IDs werden von oben nach unten mit dem Trenner :: verkettet.

4.1 Die globale UUID

Verfügt das IoT-Gerät über einen nichtflüchtigen Speicher (NVM / OTP / Secure Element), muss dort eine weltweit eindeutige UUID (nach RFC 9562) hinterlegt sein. Diese bildet die Wurzel (Root) des Adressbaums. • Syntax: UUID::[L2_Bus_ID]::[L1_Mux_ID]::[L0_Port_ID] • Beispiel (Sensor an Multiplexer auf I2C-Bus): f81d4fae-7dec-11d0-a765-00a0c91e6bf6::I2C_1::MUX_02::CH_02

4.2 API für das Adress-Routing

c typedef struct { uint8_t has_uuid; uint8_t uuid[16]; uint16_t l2_id; uint16_t l1_id; uint16_t l0_id; } giot_hardware_path_t;

typedef struct { status_t (*GetStreamByPath)(giot_hardware_path_t* path, giot_stream_t* out_stream); status_t (*ConfigureByPath)(giot_hardware_path_t* path, uint32_t config_flags); } giot_layer_interface_t; Verwende Code mit Vorsicht.

1. Autonome Topologie-Discovery

In Anlehnung an den ANSI E1.20 (RDM - Remote Device Management) Standard für serielle Netzwerke implementiert jede Ebene eine Discovery-Funktion. Dadurch kann das System zur Laufzeit dynamisch ohne statische Konfigurationstabellen gescannt werden.

5.1 Discovery-API

c typedef struct { uint16_t local_id; // Lokale Kennung auf dieser Ebene uint8_t port_type; // Stream-Quelle, Input, Output, Bi-Direktional uint8_t is_virtual; // 1 = Pass-Through, 0 = Physische Hardware char name[16]; // Lesbare Kurzbezeichnung } giot_port_info_t;

typedef struct { status_t (*ListKnownPorts)(giot_hardware_path_t* parent_path, giot_port_info_t* info_buffer, uint16_t max_entries, uint16_t* found_entries); } giot_discovery_interface_t; Verwende Code mit Vorsicht.

5.2 Industrielle Parallelen & Links

• ANSI E1.20 (RDM): Definiert die dynamische Entdeckung und Konfiguration von Ports über serielle Leitungen (Link zu ESTA / PLASA Standards). • OPC UA Node-Discovery: Hierarchische Baum-Strukturierung zur Adressierung industrieller Sensornetzwerke (Link zu OPC Foundation). Wenn du den Text in LibreOffice eingefügt hast, kann ich dir bei den nächsten Schritten helfen: • Benötigst du ein konkretes Implementierungsbeispiel in C, das zeigt, wie ein Treiber für Layer 2 (z.B. ein I2C-Bus) die ListKnownPorts-Funktion ausführt? • Sollen wir ein passendes Datenblatt-Schema (JSON/YAML) entwerfen, mit dem man ein solches Gerät vorab deklarativ beschreiben kann? In der Industrie wird die Zugriffssicherheit (Security) auf Hardware- und Schnittstellen-Ebene heute über das Prinzip „Defense in Depth“ (tiefengestaffelte Sicherheit) gelöst. Da dein Framework vom direkten Registerzugriff (Layer 0) bis zur physikalischen Einheit (Layer 4) reicht, müssen die Sicherheitsmechanismen ebenfalls auf jeder Ebene greifen. In modernen Standards wie IEC 62443 (für industrielle Automatisierung) und ISO 21434 (für Automotive Cybersecurity) wird dies durch eine Kombination aus Hardware-Sicherheitsankern und softwarebasierter Zugriffskontrolle gelöst. Hier ist die Übersicht, wie die Industrie diese Herausforderung auf die verschiedenen Schichten deines Frameworks aufteilt:

1. Hardware-Ebene (Der Vertrauensanker für Layer 0 & NVM)

Bevor Software Zugriffsrechte prüfen kann, muss die Identität des Geräts (die UUID im NVM) fälschungssicher sein. • Hardware Root of Trust (RoT): Die UUID und die kryptografischen Schlüssel werden in einem Secure Element (z. B. OPTIGA™ Trust) oder einem HSE/HSM (Hardware Security Module) auf dem Mikrocontroller abgelegt. Diese Speicher sind physisch gegen Auslesen (Side-Channel-Angriffe) geschützt. • Hardware Enforcement: Moderne CPUs nutzen MPUs (Memory Protection Units) oder Architekturen wie ARM TrustZone. Sie sperren Layer 0 (die Register) physisch für unautorisierte Software-Komponenten. Ein Applikations-Thread kann die Register von I2C_1 physisch nicht beschreiben, es sei denn, der Aufruf läuft über den autorisierten HAL-Treiber.

1. Zugriffskontrolle im HAL (Layer 1, 2 & 3)

Wenn ein Prozess über den Pfad UUID::I2C_1::MUX_A::CH_2 Daten anfordert, muss das Framework prüfen, ob dieser Prozess das überhaupt darf. • Access Control Lists (ACL) & Capability-based Security: Jeder logische Port in Layer 3 und jeder Bus in Layer 2 besitzt eine Zugriffsmatrix. Höhere Applikationen müssen beim HAL ein „Token“ oder ein kryptografisches Zertifikat vorlegen, um den Pfad freizuschalten. • Industrie-Vorbild (AUTOSAR): Der Standard nutzt Crypto Services (CSM) und Key Management. Ein Zugriff auf sicherheitskritische Busse (z. B. Bremsen-CAN im Auto) wird nur gewährt, wenn die anfragende Software-Komponente kryptografisch signiert ist.

1. Sicherheit für Shared Memory & IPC (Layer 2 Bus)

Da du Shared Memory korrekterweise als Bus in Layer 2 definiert hast, entsteht hier ein Angriffsvektor: Ein kompromittierter Prozess auf einem Kern könnte den Speicher des anderen Kerns manipulieren. • Die industrielle Lösung: MPU-Isolierung des Shared Memory. Der RAM-Bereich, der als „virtueller Bus“ dient, wird durch die Hardware-MPU strikt auf zwei Speicherseiten aufgeteilt (Core A darf nur schreiben, Core B nur lesen). • Industrie-Vorbild (OpenAMP / VirtIO): Die Signalisierung erfolgt über hardwarebasierte Mailbox-Interrupts. Daten im Puffer werden zusätzlich oft mit einer Prüfsumme (HMAC) versehen, um Manipulationen (Man-in-the-Middle im RAM) zu verhindern.

1. Datensicherheit und Integrität (Layer 3 & 4)

• Authentizität des Streams: Im Datenmodell giot_stream_t wird das Status-Byte (analog zu OPC UA) um ein Security-Feld erweitert. Sensordaten können von Layer 2 direkt im HSM signiert werden. Layer 4 transformiert die Daten nur, wenn die Signatur gültig ist. Dies verhindert, dass manipulierte Sensorwerte (z. B. gefälschte Temperaturwerte) die Anlage beschädigen.

🔑 Die wichtigsten Industriestandards für GIOT-HAL

Wenn du deine Spezifikation um das Thema Security ergänzt, solltest du dich auf diese globalen Standards beziehen und deren Links/Namen integrieren: • IEC 62443: Der weltweite Standard für Cybersecurity in der industriellen Automatisierung (Industrial IoT). Er fordert die Segmentierung von Netzwerken und Komponenten in "Zones" and "Conduits" – das lässt sich perfekt auf deine Layer-Architektur übertragen. • OPC UA Security Architecture: Löst die Zugriffssicherheit auf Applikationsebene über digitale Zertifikate (X.509). Jedes Gerät und jeder Client muss sich

authentifizieren, bevor der Adressbaum (::) ausgelesen werden darf.

🛡️ Patentsituation bei Security-Verfahren

• Kryptografische Algorithmen: Standard-Verfahren wie AES, SHA-256 oder ECC (Elliptic Curve Cryptography) sind bezüglich der mathematischen Grundlagen frei nutzbar. • Achtung bei Hardware-Implementierungen: Bestimmte Verfahren zur Beschleunigung von Krypto-Operationen in Silizium oder spezifische Protokolle zum sicheren Booten (Secure Boot) von Halbleitern (z. B. von Intel, ARM oder Apple) sind durch Patente geschützt. Für dein Software-Framework ist das jedoch unkritisch, da du nur die Schnittstelle (API) definierst, während die Chiphersteller die patentierten Bausteine darunter liefern. Möchtest du, dass wir für das finale Dokument ein dediziertes Kapitel „6. Security & Access Control“ hinzufügen, das genau definiert, wie ein Prozess seine Rechte nachweisen muss, bevor er eine giot_hardware_path_t aufrufen darf? Hier ist das dedizierte Security-Kapitel für deine Spezifikation. Es ist so konzipiert, dass es die bestehenden Routinen moderner Standards wie AUTOSAR (SecOC), OPC UA und IEC 62443 nativ abbildet. Du kannst diesen Text direkt als „Kapitel 6“ in dein LibreOffice-Dokument einfügen.

1. Security & Access Control Framework

Um die Konformität mit den globalen Cybersecurity-Standards IEC 62443 (Industrial IoT) und ISO 21434 / AUTOSAR (Automotive) zu garantieren, integriert GIOT-HAL ein tiefengestaffeltes Sicherheitskonzept (Defense in Depth). Der Zugriff auf Hardware-Pfade (giot_hardware_path_t) wird strikt reglementiert und kryptografisch abgesichert.

6.1 Hardware Root of Trust (Layer 0 & NVM Security)

Die eindeutige Identifikation und Verifizierung des Geräts basiert auf dedizierten Hardware-Sicherheitsankern: • UUID Protection: Die im NVM hinterlegte globale UUID muss in einem geschützten Speicherbereich (OTP-Flit, Secure Element oder Hardware Security Module / HSM) abgelegt sein. Sie ist unveränderlich und dient als Basis für die Ableitung gerätespezifischer kryptografischer Schlüssel. • Hardware Isolation: Der direkte Registerzugriff in Layer 0 muss über Hardware-Mechanismen wie eine Memory Protection Unit (MPU) oder isolierte Ausführungsumgebungen (z. B. ARM TrustZone) abgesichert sein. Unautorisierte Applikations-Threads werden bei direktem Registerzugriff hardwareseitig blockiert.

6.2 Token-basierte Zugriffskontrolle (Layer 1 bis 3)

Um bestehende industrielle Routinen zur Rechteverwaltung nahtlos zu übernehmen, nutzt GIOT-HAL ein Capability-basiertes Token-Modell. Höhere Softwarekomponenten müssen bei jedem geschützten API-Aufruf ein Sicherheits-Token übergeben. c // Struktur für den autorisierten Zugriffskontext typedef struct { uint32_t client_id; // Eindeutige Kennung der anfragenden Software-Komponente uint32_t permission_mask; // Bitmaske für erlaubte Operationen (READ, WRITE, CONFIG, DISC) uint8_t token_signature[32]; // Kryptografischer MAC / Signatur des Tokens (z. B. HMACS-SHA256) } giot_security_context_t; Verwende Code mit Vorsicht. Die bestehenden funktionalen Schnittstellen werden um diesen Sicherheitskontext erweitert: c typedef struct { // Erweiterte Schnittstelle mit integrierter Rechteprüfung status_t (*GetSecureStreamByPath)(giot_hardware_path_t* path, giot_security_context_t* sec_ctx, giot_stream_t* out_stream); } giot_secure_layer_interface_t; Verwende Code mit Vorsicht. • Routinen-Kompatibilität: Dieses

Modell spiegelt exakt das Verhalten von OPC UA Access Control Lists (ACL) wider. Das Framework vergleicht bei einem Aufruf von UUID::I2C_1::MUX_A::CH_2 die client_id und die Signatur des Tokens mit den im HAL hinterlegten Zugriffsrechten der jeweiligen Hardware-Zone.

6.3 Secure On-Board Communication (Layer 2 Bus & Shared Memory)

Wenn Daten über Shared Memory oder serielle Busse transportiert werden, greifen etablierte Mechanismen gegen Manipulation (Man-in-the-Middle im RAM) und Replay-Angriffe: • AUTOSAR SecOC Konformität: Datenströme (giot_stream_t), die über kritische Layer-2-Busse laufen, werden durch das HSM mit einem standardisierten Freshness Counter (gegen Replay-Angriffe) und einem Authentisierungs-Code (MAC) versehen. • Shared-Memory-Isolierung: Bei Multiprozessor-Systemen (AMP) wird der als Bus definierte Shared-Memory-Bereich über die System-MPU hardwareseitig isoliert. Ein Kern besitzt strikte Write-Only-Rechte, der Kommunikationspartner exklusive Read-Only-Rechte. Die Signalisierung erfolgt über hardwareseitig geschützte Inter-Processor-Interrupts (Mailboxen).

6.4 Datenintegrität auf Anwendungsebene (Layer 4)

Layer 4 führt die physikalische Transformation von Datenströmen nur dann aus, wenn das im Datenmodell integrierte Status-Byte Validität signalisiert. Das im Kapitel 3 definierte quality_status-Byte wird dafür um Security-Flags erweitert (z. B. STATUS_UNAUTHENTICATED). Schlägt die Verifizierung in Layer 2 oder Layer 3 fehl, wird der Stream als ungültig markiert und von Layer 4 blockiert, um das Einspeisen manipulierter Sensorwerte in die Applikationslogik zu verhindern.

6.5 Konforme Standards & Referenz-Links

• ISA/IEC 62443: Definiert die Anforderungen an die komponentenbasierte Sicherheit und die Aufteilung in Sicherheitszonen innerhalb von IoT- und Automatisierungssystemen. • AUTOSAR Secure On-Board Communication (SecOC): Liefert die industriellen Spezifikationen und Routinen zur Verifizierung von Datenströmen auf Bus-Ebene mittels MACs und Freshness-Managern. • OPC UA Security Architecture: Referenz für die hierarchische Benutzer- und Applikations-Authentifizierung beim Zugriff auf logische Adressbäume. Das Sicherheitskapitel fügt sich nahtlos ein. Um dein Dokument final abzurunden, schlage ich vor: • Sollen wir ein prägnantes Anwendungsbeispiel (Use-Case) anhängen, das den kompletten Ablauf – von der Erkennung eines Geräts über die Rechteprüfung bis zur Datentransformation – in einem Fließtext beschreibt? • Oder fehlen dir noch spezifische Fehlercodes (Status-Returns), die das Framework im Falle einer Sicherheitsverletzung zurückgeben soll? Die Ergänzung um diese Metadaten-Deklaration bringt eine entscheidende Stärke in das Framework: Es bleibt absolut herstellerunabhängig und transparent. Indem der HAL nicht die Inhalte (Payloads) selbst vorschreibt, sondern stattdessen die Metadaten-Kennzeichnung des verwendeten Payload-Standards erzwingt, können bestehende Protokolle (wie CANopen, J1939, Zigbee Cluster Library oder proprietäre Datenstrukturen) unverändert durch GIOT-HAL tunneln. Hier ist die detaillierte Spezifikation für diesen Mechanismus, bereit zum Einfügen in dein LibreOffice-Dokument (z. B. als Ergänzung zu Kapitel 3 für Daten und Kapitel 6 für Fehler):

1. Inhaltstransparenz und Payload-Standardisierung

GIOT-HAL verzichtet bewusst darauf, die semantische Struktur des transportierten Inhalts (Payload) innerhalb des Datenstroms starr vorzuschreiben. Stattdessen implementiert das Framework ein Protokoll-Indikator-Modell. Jedes Datenstrom-Objekt und jede Fehlermeldung muss explizit deklarieren, nach welchem industriellen oder herstellerspezifischen Standard der Inhalt codiert ist.

5.1 Erweitertes Datenmodell mit Content-Standard-Indikator

Das in Kapitel 3 definierte Datenmodell (giot_stream_t) wird um ein standardisiertes Feld zur Protokoll-Identifikation erweitert: c // Registrierte globale Standard-IDs für Payloads typedef enum { PAYLOAD_STD_RAW_DIGITS = 0x0000, // Reine, herstellerspezifische ADC/DAC-Rohwerte PAYLOAD_STD_SI_UNITS = 0x0001, // Standardisierte SI-Einheiten nach Layer 4 PAYLOAD_STD_CANOPEN_OD = 0x0010, // CANopen Object Dictionary Profil PAYLOAD_STD_SAE_J1939 = 0x0011, // SAE J1939 Parameter Group Number (PGN) PAYLOAD_STD_ZIGBEE_ZCL = 0x0020, // Zigbee Cluster Library Datenstruktur PAYLOAD_STD_BLE_GATT = 0x0030, // Bluetooth LE GATT Charakteristik PAYLOAD_STD_PROP_VENDOR = 0xFFFF // Proprietärer Hersteller-Standard } giot_payload_std_t;

typedef struct { void* buffer_ptr; // Speicheradresse (RAM, DMA oder Shared-Memory) uint32_t element_count; // Anzahl der Datenpunkte uint32_t element_size; // Größe eines Datenpunktes in Bytes uint64_t timestamp; // Globaler Zeitstempel der Erfassung uint8_t quality_status; // Status-Byte (Konform zu OPC UA) giot_payload_std_t payload_standard;// DER ERZWUNGENE STANDARD-INDIKATOR uint16_t payload_profile; // Optionale Sub-Profil-ID (z. B. CANopen Profil-Nr.) } giot_stream_t; Verwende Code mit Vorsicht. Durch diese Struktur weiß eine höherlagige Applikation oder ein Parser in Layer 4 beim Lesen des Streams sofort, wie das Byte-Array im buffer_ptr zu interpretieren ist, ohne dass der HAL das Protokoll selbst verstehen muss.

1. Standardisierte Fehlerbehandlung (Error Handling)

Das Prinzip der Protokoll-Deklaration gilt gleichermaßen für Fehlermeldungen. Da hardwarenahe Fehler (z. B. Busfehler auf Layer 2 oder Sensorfehler auf Layer 3) je nach Industrie extrem unterschiedlich codiert werden, erzwingt GIOT-HAL auch hier die Angabe des zugrundeliegenden Fehler-Standards.

7.1 Die universelle Fehler-Struktur

Jede Funktion im Framework gibt einen standardisierten Status-Typen zurück. Im Falle eines Fehlers liefert das System eine detaillierte Fehler-Struktur, die den Ursprungs-Standard deklariert: c // Registrierte Standard-IDs für die Fehler-Codierung typedef enum { ERR_STD_GIOT_NATIVE = 0x00, // Interner, nativer GIOT-HAL Fehlercode ERR_STD_AUTOSAR_DEM = 0x01, // AUTOSAR Diagnostic Event Manager (DTC) ERR_STD_CANOPEN_EMCY = 0x02, // CANopen Emergency Code (EMCY) ERR_STD_OPCUA_STATUS = 0x03, // OPC UA StatusCode ERR_STD_I2C_SPECIFIC = 0x10, // Spezifischer Hardware-Fehler des I2C-Bus-Protokolls ERR_STD_VENDOR_SPECIFIC = 0xFF // Proprietärer Fehlercode des Halbleiter-Herstellers } giot_error_std_t;

typedef struct { status_t giot_status; // Grobe native Einordnung (z. B. GIOT_ERR_SECURITY, GIOT_ERR_IO) giot_error_std_t error_standard; // DER ERZWUNGENE FEHLER-STANDARD-INDIKATOR uint32_t specific_code; // Der echte, unveränderte Fehlercode aus dem jeweiligen Standard } giot_error_context_t; Verwende Code mit Vorsicht.

7.2 Praxis-Beispiel für ein Fehler-Szenario

Tritt auf einem CAN-Bus in Layer 2 ein kritischer Fehler auf, weil ein Sensor nicht antwortet, kann der Layer-2-Treiber den originalen industriellen Fehlercode eins zu eins durchreichen: • giot_status = GIOT_ERR_HARDWARE_FAILURE • error_standard = ERR_STD_CANOPEN_EMCY • specific_code = 0x3000 (Industrieller CANopen-Code für "Hardware Error") Höher aufliegende Diagnose-Software (z. B. ein Cloud-Gateway) kann diesen Wert ohne Informationsverlust direkt in das bestehende Industrie-Überwachungssystem einspeisen.

🔗 Industrielle Parallelen & offizielle Standards

• SAE J1939 & CANopen: Diese Protokolle nutzen exakt dieses Prinzip. Sie definieren "Profiles" und "Parameter Groups", überlassen die bitgenaue Auslegung der Hardware-Schnittstelle jedoch der Transportschicht. • AUTOSAR DEM (Diagnostic Event Manager): Der führende Standard in der Automobilindustrie speichert Fehler als standardisierte Diagnostic Trouble Codes (DTC) (Link zu AUTOSAR Classic). GIOT-HAL ermöglicht es über das giot_error_context_t-Modell, diese DTCs nativ und unverfälscht zu transportieren. Mit dieser Ergänzung deckt deine Spezifikation nun auch die Interoperabilität für bestehende Protokolle perfekt ab. Wenn du möchtest, können wir als Nächstes: • Die vollständige, konsolidierte Fassung aller Kapitel in einem einzigen, großen Block ausgeben, damit du das Dokument in LibreOffice sauber abspeichern kannst. • Oder wir definieren die nativen GIOT-Grundfehlercodes (status_t), die jeder Treiber mindestens unterstützen muss. Hier ist die vollständige, konsolidierte Gesamtspezifikation inklusive aller definierten Layer, des Stream-Modells, der Hierarchie, der RDM-Discovery, der Security-Architektur nach IEC 62443 / AUTOSAR sowie der neuen Content- und Fehler-Metadaten inklusive der nativen Fehlercodes (status_t). Du kannst diesen gesamten Block kopieren und direkt in LibreOffice Writer einfügen.

SPEZIFIKATION: GLOBAL IOT HARDWARE ABSTRACTION LAYER (GIOT-HAL)

Dokumenten-Version: 1.0.0 Status: Standard-Entwurf Zielgruppe: Entwickler von Halbleitern, IoT-Betriebssystemen (RTOS) und Edge-Geräten

1. Einleitung und Zielsetzung

Der Global IoT Hardware Abstraction Layer (GIOT-HAL) definiert ein universelles, modulares Schnittstellen-Framework für eingebettete Systeme und IoT-Geräte. Das primäre Ziel ist die vollständige Entkopplung der Anwendungslogik von der darunterliegenden physischen Hardware-Topologie, den Bus-Strukturen und Halbleiter-Registern. Durch ein striktes 5-Schichten-Modell, kombiniert mit einer hierarchischen Adressierung, tiefengestaffelter Security und autonomen Discovery-Mechanismen, ermöglicht GIOT-HAL eine echte Hardware-Agnosie. Applikationen können ohne Code-Änderung von einem einfachen 8-Bit-Mikrocontroller auf ein heterogenes Multi-Core-Schnittstellen-Gateway portiert werden.

1. Kernarchitektur & Schichtenmodell (Layer 0 bis 4)

Das Framework ist vertikal in fünf logische Ebenen unterteilt. Jede Ebene implementiert eine standardisierte Schnittstelle, um auf alle tieferen Ebenen zugreifen zu können (Tunneling). +----------+

| Layer 4: Physical Unit Transformation (Skalierung/Metrik)
| +----------+

| Layer 3: External Abstract Interface (Logischer Port)
| +----------+

| Layer 2: Bus-Topology & IPC Controller (I2C, SPI, ShMem)
| +----------+

| Layer 1: Port Expander / Multiplexer Layer
| +----------+

| Layer 0: Direct CPU Register Access (MCAL / Local I/O)
| +----------+

2.1 Layer 0: Direct CPU Register Access

• Definition: Die hardwarenaheste Schicht. Sie besitzt direkten, exklusiven Lese- und Schreibzugriff auf die speichergemappten Register (Memory-Mapped I/O) der CPU/MCU und steuert die On-Chip-Peripherie (interne GPIOs, interne ADCs). • Industrie-Äquivalente: • ARM CMSIS-Core: Standardisierter Registerzugriff für ARM Cortex-M Prozessoren (Link zu ARM CMSIS). • AUTOSAR Classic MCAL: Microcontroller Abstraction Layer (Link zu AUTOSAR Classic). • Patentsituation: Registerzugriffe und Memory-Mapping sind gemeinfreier Stand der Technik.

2.2 Layer 1: Port Expander / Multiplexer Layer

• Definition: Verwaltet Topologien, bei denen ein Pin oder Port durch externe Logikbausteine (z. B. Bit-Shifter, analoge/digitale Multiplexer) vervielfacht wird. Wenn ein Pin direkt an der CPU hängt, ist dieser Layer leer (Transparent-Pass-Through), muss aber die standardisierte API-Signatur bedienen. • Industrie-Äquivalente: • Linux pinctrl/GPIO Subsystem: Kernel-Treiber für GPIO-Multiplexing. • Chiisai HAL: Ein offener Standard für ressourcenarme MCUs. • Patentsituation: Grundlegendes Multiplexing ist gemeinfrei. Dynamische, laufzeitabhängige Auto-Routing-Matrix-Verfahren stehen teilweise unter Herstellerpatent (z. B. FPGA-Sektoren).

2.3 Layer 2: Bus-Topology & IPC Controller

• Definition: Kapselt Datenübertragungen, die über serielle/parallele Busprotokolle (I2C, SPI, CAN) oder Inter-Processor Communication (Shared Memory / Mailboxen) laufen. Schicht 2 verarbeitet zudem alle Hardware-Interrupts, um z. B. Daten aus Sensor-FIFOs asynchron in Datenströme zu laden. • Industrie-Äquivalente: • Embedded-HAL (Rust): Plattformunabhängige Bus-Traits (Link zu Embedded-HAL). • OpenAMP: Standard für asymmetrisches Multi-Processing via Shared Memory (Link zu OpenAMP). • Patentsituation: Standard-Busse (I2C, SPI) sind frei. Komplexe Busse wie I3C (MIPI Alliance) oder proprietäre Automotive-Protokolle enthalten patentierte IP-Pools (FRAND-Lizenzierung erforderlich).

2.4 Layer 3: External Abstract Interface

• Definition: Die rein logische Sicht auf das IoT-Gerät. Es existieren keine Register oder Busse mehr, sondern nur noch logische Datenpunkte (z. B. HEATER_CONTROL). Schicht 3 abstrahiert die physische Herkunft vollständig. • Industrie-Äquivalente: • Arduino-Core-API: Abstraktionen wie analogRead() / digitalWrite(). • Android OS Hardware Abstraction Layer (HAL): Vereinheitlicht Hardware-Aufrufe via AIDL (Link zu Android HAL). • Patentsituation: Reine logische API-Mappings sind Entwurfsmuster und nicht patentierbar.

2.5 Layer 4: Physical Unit Transformation

• Definition: Wandelt die herstellerspezifischen Rohdaten (Bits/Digits) in standardisierte, physikalische SI-Einheiten (Volt, Celsius, Pascal) um. Führt Linearisierungen und Kalibrierungen durch. • Industrie-Äquivalente: • Zephyr RTOS Sensor API: Liefert transformierte Datenstrukturen (Link zu Zephyr Sensor API). • Patentsituation: Einfache mathematische Skalierung ist frei. Komplexe, softwarebasierte Kompensationsalgorithmen (z. B. Machine-Learning-gestützte Drift-Kompensation bei Gassensoren) sind häufig durch Sensorhersteller patentrechtlich geschützt.

1. Datenmodell: "Stream-as-a-Value" und Inhaltstransparenz

Um High-Speed-Sensoren (Audio, Vibration) und Einzelwert-Sensoren (Temperatur) identisch zu behandeln, definiert GIOT-HAL jeden Datenpunkt nativ als Stream. Ein Einzelwert ist architektonisch ein Stream mit der Länge 1. GIOT-HAL verzichtet bewusst darauf, die semantische Struktur des transportierten Inhalts (Payload) innerhalb des Datenstroms starr vorzuschreiben. Stattdessen implementiert das Framework ein Protokoll-Indikator-Modell. Jedes Datenstrom-Objekt muss explizit deklarieren, nach welchem industriellen oder herstellerspezifischen Standard der Inhalt codiert ist. c // Registrierte globale Standard-IDs für Payloads typedef enum { PAYLOAD_STD_RAW_DIGITS = 0x0000, // Reine, herstellerspezifische ADC/DAC-Rohwerte PAYLOAD_STD_SI_UNITS = 0x0001, // Standardisierte SI-Einheiten nach Layer 4 PAYLOAD_STD_CANOPEN_OD = 0x0010, // CANopen Object Dictionary Profil PAYLOAD_STD_SAE_J1939 = 0x0011, // SAE J1939 Parameter Group Number (PGN) PAYLOAD_STD_ZIGBEE_ZCL = 0x0020, // Zigbee Cluster Library Datenstruktur PAYLOAD_STD_BLE_GATT = 0x0030, // Bluetooth LE GATT Charakteristik PAYLOAD_STD_PROP_VENDOR = 0xFFFF // Proprietärer Hersteller-Standard } giot_payload_std_t;

typedef struct { void* buffer_ptr; // Speicheradresse (RAM, DMA oder Shared-Memory) uint32_t element_count; // Anzahl der Datenpunkte (Einzelwert = 1) uint32_t element_size; // Größe eines Datenpunktes in Bytes uint64_t timestamp; // Globaler Zeitstempel der Erfassung uint8_t quality_status; // Status-Byte (Konform zu OPC UA: Valid, Error, Degraded) giot_payload_std_t payload_standard; // DER ERZWUNGENE STANDARD-INDIKATOR uint16_t payload_profile; // Optionale Sub-Profil-ID (z. B. CANopen Profil-Nr.) } giot_stream_t; Verwende Code mit Vorsicht.

1. Hierarchisches Adressierungssystem

Jedes Element im IoT-Gerät ist über einen eindeutigen Pfad adressierbar. Die Schichten-IDs werden von oben nach unten mit dem Trenner :: verkettet.

4.1 Die globale UUID

Verfügt das IoT-Gerät über einen nichtflüchtigen Speicher (NVM / OTP / Secure Element), muss dort eine weltweit eindeutige UUID (nach RFC 9562) hinterlegt sein. Diese bildet die Wurzel (Root) des Adressbaums. • Syntax: UUID::[L2_Bus_ID]::[L1_Mux_ID]::[L0_Port_ID] • Beispiel (Sensor an Multiplexer auf I2C-Bus): f81d4fae-7dec-11d0-a765-00a0c91e6bf6::I2C_1::MUX_02::CH_02

4.2 API für das Adress-Routing

c typedef struct { uint8_t has_uuid; uint8_t uuid[16]; uint16_t l2_id; uint16_t l1_id; uint16_t l0_id; } giot_hardware_path_t;

typedef struct { status_t (*GetStreamByPath)(giot_hardware_path_t* path, giot_stream_t* out_stream); status_t (*ConfigureByPath)(giot_hardware_path_t* path, uint32_t config_flags); } giot_layer_interface_t; Verwende Code mit Vorsicht.

1. Autonome Topologie-Discovery

In Anlehnung an den ANSI E1.20 (RDM - Remote Device Management) Standard für serielle Netzwerke implementiert jede Ebene eine Discovery-Funktion. Dadurch kann das System zur Laufzeit dynamisch ohne statische Konfigurationstabellen gescannt werden.

5.1 Discovery-API

c typedef struct { uint16_t local_id; // Lokale Kennung auf dieser Ebene uint8_t port_type; // Stream-Quelle, Input, Output, Bi-Direktional uint8_t is_virtual; // 1 = Pass-Through, 0 = Physische Hardware char name[16]; // Lesbare Kurzbezeichnung } giot_port_info_t;

typedef struct { status_t (*ListKnownPorts)(giot_hardware_path_t* parent_path, giot_port_info_t* info_buffer, uint16_t max_entries, uint16_t* found_entries); } giot_discovery_interface_t; Verwende Code mit Vorsicht.

5.2 Industrielle Parallelen & Links

• ANSI E1.20 (RDM): Definiert die dynamische Entdeckung und Konfiguration von Ports über serielle Leitungen (Link zu ESTA / PLASA Standards). • OPC UA Node-Discovery: Hierarchische Baum-Strukturierung zur Adressierung industrieller Sensornetzwerke (Link zu OPC Foundation).

1. Security & Access Control Framework

Um die Konformität mit den globalen Cybersecurity-Standards IEC 62443 (Industrial IoT) und ISO 21434 / AUTOSAR (Automotive) zu garantieren, integriert GIOT-HAL ein tiefengestaffeltes Sicherheitskonzept (Defense in Depth). Der Zugriff auf Hardware-Pfade wird strikt reglementiert.

6.1 Hardware Root of Trust & Isolation

• UUID Protection: Die im NVM hinterlegte globale UUID muss in einem geschützten Speicherbereich (OTP, Secure Element oder HSM) abgelegt sein. • Hardware Isolation: Der direkte Registerzugriff in Layer 0 muss über MPU- oder TrustZone-Mechanismen abgesichert sein. Unautorisierte Threads werden hardwareseitig blockiert.

6.2 Token-basierte Zugriffskontrolle (Layer 1 bis 3)

Höhere Softwarekomponenten müssen bei jedem geschützten API-Aufruf ein Sicherheits-Token

übergeben. Das spiegelt exakt das Verhalten von OPC UA Access Control Lists (ACL) wider. c typedef struct { uint32_t client_id; // Eindeutige Kennung der anfragenden Komponente uint32_t permission_mask; // Bitmaske für erlaubte Operationen (READ, WRITE, etc.) uint8_t token_signature[32]; // Kryptografischer MAC / Signatur des Tokens (HMAC-SHA256) } giot_security_context_t;

typedef struct { status_t (*GetSecureStreamByPath)(giot_hardware_path_t path, giot_security_context_t* sec_ctx, giot_stream_t* out_stream); } giot_secure_layer_interface_t; Verwende Code mit Vorsicht.

6.3 Secure On-Board Communication & Integrity

• AUTOSAR SecOC Konformität: Datenströme (giot_stream_t) auf kritischen Bussen werden durch das HSM mit einem standardisierten Freshness Counter und einem Authentisierungs-Code (MAC) versehen. • Shared-Memory-Isolierung: Der als Bus definierte Shared-Memory-Bereich (AMP) wird über die System-MPU in strikte Write-Only und Read-Only Partitionen aufgeteilt. • Data Integrity (Layer 4): Schlägt die Verifizierung fehl, wird der Datenstrom mit einem Security-Flag im quality_status-Byte markiert (z. B. STATUS_UNAUTHENTICATED) und von Layer 4 blockiert.

6.4 Konforme Standards & Referenz-Links

• ISA/IEC 62443: Standard für komponentenbasierte Sicherheit und Aufteilung in Sicherheitszonen. • AUTOSAR Secure On-Board Communication (SecOC): Spezifikation zur Verifizierung von Datenströmen auf Bus-Ebene mittels MACs.

1. Native Fehlerbehandlung und Fehler-Metadaten

GIOT-HAL erzwingt analog zur Payload die explizite Deklaration des zugrundeliegenden Fehler-Standards. Jede Funktion im Framework gibt einen standardisierten, nativen Status-Typen (status_t) zurück und befüllt bei Fehlern den erweiterten Kontext.

7.1 Native GIOT-Fehlercodes (status_t)

c typedef enum { GIOT_SUCCESS = 0x00, // Operation erfolgreich durchgeführt GIOT_ERR_NOT_INITIALIZED = 0x01, // Layer oder Peripherie nicht initialisiert GIOT_ERR_INVALID_PATH = 0x02, // Der angegebene Hardware-Pfad existiert nicht GIOT_ERR_BUFFER_TOO_SMALL = 0x03, // Bereitgestellter Speicherpuffer ist zu klein GIOT_ERR_HARDWARE_FAILURE = 0x04, // Physischer Hardware- oder Bus-Fehler detektiert GIOT_ERR_TIMEOUT = 0x05, // Zeitüberschreitung bei Bus- oder Registerzugriff GIOT_ERR_SECURITY_DENIED = 0x06, // Zugriff verweigert (Token ungültig / unzureichende Rechte) GIOT_ERR_BUSY = 0x07, // Ressource ist durch eine andere Operation blockiert GIOT_ERR_UNSUPPORTED_STD = 0x08, // Angeforderter Payload- oder Error-Standard nicht unterstützt GIOT_ERR_UNKNOWN = 0xFF // Unbekannter oder nicht klassifizierter Fehler } status_t; Verwende Code mit Vorsicht.

7.2 Universelle Fehler-Struktur (Error Context)

c // Registrierte Standard-IDs für die Fehler-Codierung typedef enum { ERR_STD_GIOT_NATIVE = 0x00, // Interner, nativer GIOT-HAL Fehlercode ERR_STD_AUTOSAR_DEM = 0x01, // AUTOSAR Diagnostic Event Manager (DTC) ERR_STD_CANOPEN_EMCY = 0x02, //

CANopen Emergency Code (EMCY) ERR_STD_OPCUA_STATUS = 0x03, // OPC UA StatusCode ERR_STD_I2C_SPECIFIC = 0x10, // Spezifischer Hardware-Fehler des I2C-Protokolls ERR_STD_VENDOR_SPECIFIC = 0xFF // Proprietärer Fehlercode des Halbleiter-Herstellers } giot_error_std_t;

typedef struct { status_t giot_status; // Native Einordnung (siehe 7.1) giot_error_std_t error_standard; // DER ERZWUNGENE FEHLER-STANDARD-INDIKATOR uint32_t specific_code; // Der originale Fehlercode aus dem Zielstandard } giot_error_context_t; Verwende Code mit Vorsicht. Tritt beispielsweise ein Fehler im AUTOSAR-Verbund auf, kann das System giot_status = GIOT_ERR_SECURITY_DENIED, error_standard = ERR_STD_AUTOSAR_DEM und als specific_code den exakten fahrzeugspezifischen Diagnostic Trouble Code (DTC) durchreichen. Die Spezifikation ist nun als umfassendes Dokument fertiggestellt. Wenn du möchtest, können wir das Projekt wie folgt weiter vertiefen: • Sollen wir eine formale Grammatik für den Adresspfad definieren, falls du einen Parser (z.B. in C++ oder Rust) dafür schreiben musst? • Benötigst du ein Beispiel-Datenblatt (Manifest-Datei in JSON/YAML), mit dem Halbleiterhersteller ihre Chips GIOT-HAL-konform beschreiben können? Hier sind die formale Grammatik für den Adresspfad, die Manifest-Spezifikation (YAML/JSON) sowie das vollständige, integrierte Praxisbeispiel für dein Dokument.

1. Formale Pfad-Grammatik (ABNF)

Um das Parsing der hierarchischen Hardware-Pfade auf Gateways oder innerhalb von Software-Stacks zu standardisieren, wird die Syntax nach RFC 5234 (Augmented BNF) definiert. abnf ; GIOT-HAL Address Path Grammar giot-path = [uuid "::"] l2-id "::" l1-id "::" l0-id

uuid = 8HEXDIG "-" 4HEXDIG "-" 4HEXDIG "-" 4HEXDIG "-" 12HEXDIG ; Konform zu RFC 9562 (UUID v4/v5)

l2-id = identifier ; Bus-System, IPC-Kanal oder Shared-Memory-Name

l1-id = identifier ; Multiplexer-, Expander-ID oder "PASSTHROUGH"

l0-id = identifier ; Physische Pin-Nummer, Kanal oder Register-Offset

identifier = 1*64(ALPHA / DIGIT / "_") ; Alphanumerischer Kurz-String, max. 64 Zeichen Verwende Code mit Vorsicht.

1. GIOT-HAL Manifest-Spezifikation (Deklaratives Datenblatt)

Halbleiterhersteller und IoT-Systemintegratoren müssen ihre Hardware-Topologie in Form einer standardisierten Manifest-Datei (YAML oder ressourcensparendes JSON) bereitstellen. Dies erlaubt es Compilern oder Laufzeit-Engines, den HAL-Baum automatisch zu generieren.

9.1 YAML-Manifest-Schema (giot_manifest.yaml)

yaml giot_manifest_version: "1.0.0" device: vendor: "IoT-Silicon-Corp" model: "SecureNode-X4" nvm_has_uuid: true

Topologie-Beschreibung von unten nach oben (L0 bis L2)

topology: layer2_busses: - id: "I2C_1" type: "I2C" base_register: "0x40005400" # Layer 0 Zuordnung interrupt_vector: 31 security_zone: "SECURE_ZONE_A"

```
  layer1_devices:
    - id: "MUX_02"
      type: "PCA9548_8ChannelMux"
      bus_address: "0x70"

      layer0_ports:
        - id: "CH_02"
          type: "ANALOG_INPUT"
          stream_capable: true
          payload_standard: "PAYLOAD_STD_RAW_DIGITS"

- id: "SHMEM_COMM_0"
  type: "SHARED_MEMORY"
  base_register: "0x20040000" # Startadresse des Dual-Port-RAM
  buffer_size_bytes: 4096
  security_zone: "HIGH_SECURITY_ZONE"
  layer1_devices:
    - id: "PASSTHROUGH"
      layer0_ports:
        - id: "TELEMETRY_STREAM"
          type: "DATA_STREAM"
          stream_capable: true
          payload_standard: "PAYLOAD_STD_SI_UNITS"
```

Verwende Code mit Vorsicht.

1. End-to-End Praxisbeispiel (Use-Case)

Das folgende Szenario beschreibt den vollständigen, sequenziellen Ablauf von der Geräte-Erkennung über die Rechteprüfung bis hin zum gestreamten Datenempfang.

Szenario: Hochgeschwindigkeits-Vibrationssensor im Industrial IoT

Ein industrieller Schwingungssensor überwacht ein Lager in einer Produktionsanlage. Er hängt an Kanal 2 (CH_02) des Multiplexers MUX_02, welcher über den I2C-Bus I2C_1 angebunden ist. Die Auswertung erfolgt auf einem Edge-Gateway.

Schritt 1: Topologie-Discovery (RDM-Modus)

Beim Systemstart sendet das Edge-Gateway einen Discovery-Aufruf an das IoT-Gerät.

1. Das Gateway ruft ListKnownPorts auf Layer 2 auf. Layer 2 meldet die Existenz von I2C_1.
2. Das Gateway kaskadiert den Aufruf tiefer: ListKnownPorts für den Pfad UUID::I2C_1 liefert den Multiplexer MUX_02.
3. Der finale Scan liefert den Port CH_02. Das Gateway weiß nun ohne vorherige Programmierung, dass unter dem Pfad f81d4fae-7dec-11d0-a765-00a0c91e6bf6::I2C_1::MUX_02::CH_02 ein Vibrationsdatenstrom existiert.

Schritt 2: Token-Verifizierung & Security-Check

Die Analyse-Applikation (Client-ID: 0x99AA) möchte den Stream abonnieren.

1. Die Applikation generiert ein Krypto-Token, signiert mit dem Anlagenschlüssel via HMAC-SHA256, und ruft GetSecureStreamByPath auf Layer 3 auf.
2. Layer 3 validiert das Token im Security Framework (Kapitel 6). Die permission_mask erlaubt das Lesen (READ).
3. Die MPU schaltet den Speicherbereich des I2C-1-Buscontrollers exklusiv für diese Transaktion frei.

Schritt 3: Asynchroner Datentransport (Layer 2 & 1)

Der Sensor löst einen Hardware-Interrupt aus, da sein interner FIFO-Puffer voll ist.

1. Interrupt-Verarbeitung auf Layer 2: Der Interrupt schlägt direkt in Layer 2 (Bus-Controller) auf. Layer 2 stoppt sofort alle Hintergrundprozesse und aktiviert den DMA-Kanal.
2. Kanal-Routing auf Layer 1: Layer 2 liest aus dem Manifest, dass das Signal über MUX_02 geroutet ist. Layer 1 setzt blitzschnell das Routing-Register via Layer 0, um Kanal 2 exklusiv zu schalten.
3. Die Rohdaten (1024 Vibrations-Messpunkte) werden via I2C-DMA direkt in den RAM-Bereich des giot_stream_t-Objekts geschrieben. Der payload_standard wird auf PAYLOAD_STD_RAW_DIGITS gesetzt.

Schritt 4: Physikalische Transformation (Layer 4)

Der rohe Stream wird an Layer 4 übergeben, um ihn für die Anwendung nutzbar zu machen.

1. Layer 4 prüft das quality_status-Byte. Da keine EMV-Störungen oder Security-Verletzungen vorlagen, steht dieses auf STATUS_VALID.
2. Layer 4 liest den Payload-Typ (PAYLOAD_STD_RAW_DIGITS). Er holt die Kalibrierungsdaten aus dem Flash-Speicher und berechnet aus den Digits die reale Beschleunigung in m/s^2.
3. Das Objekt wird modifiziert: Der Puffer enthält nun berechnete Gleitkommawerte (float), und das Feld payload_standard wird auf PAYLOAD_STD_SI_UNITS umgeschrieben. Die Applikation erhält einen mathematisch korrekten, verifizierten Datenstrom. Damit ist deine Spezifikation von der Theorie bis hin zum realen Code- und Daten-Beispiel vollständig. Wenn du diesen Entwurf nun fertigstellst: • Sollen wir ein passendes C-Code-Snippet für den Pfad-Parser schreiben, der den String UUID::L2::L1::L0 zerlegt und validiert? • Oder möchtest du das Dokument mit einem Glossar der verwendeten Begriffe (NVM, MPU, HSM, RDM, etc.) für LibreOffice abschließen? Hier sind die beiden gewünschten Abschnitte, exakt im Markdown-Format formatiert, sodass du sie nahtlos als Kapitel 11 und Kapitel 12 an das Ende deines LibreOffice-Dokuments anhängen kannst.
4. Relevante Industriestandards und regulatorische Vorgaben

Für die weltweite Standardisierung und Zertifizierung eines GIOT-HAL-konformen IoT-Geräts müssen die folgenden internationalen Standards zwingend beachtet und in den jeweiligen Layern

implementiert werden:

11.1 Security & Cybersecurity Standards

• ISA/IEC 62443: Der globale Kernstandard für IT/OT-Sicherheit in industriellen Automatisierungssystemen. Er definiert die Anforderungen an "Secure-by-Design"-Komponenten und die logische Zonentrennung (wichtig für die Absicherung der Layer 0–3). • ISO/SAE 21434: Cybersecurity-Standard für die Automobilindustrie. Er reguliert das Risikomanagement über den gesamten Lebenszyklus und ist die rechtliche Basis für die im Automotive-Sektor genutzten Hardware-Sicherheitsroutinen. • AUTOSAR Secure On-Board Communication (SecOC): Spezifiziert die bitgenaue Einbettung von kryptografischen Prüfsummen (MACs) und Freshness-Countern direkt in die Datenströme der Bus-Systeme (Layer 2). • NIST SP 800-193: Richtlinie für "Platform Firmware Resiliency". Sie beschreibt, wie Hardware (Layer 0) und NVM-Inhalte vor unbefugter Modifikation und Injektionen geschützt werden müssen.

11.2 Geräte-Discovery & Daten-Modellierung

• ANSI E1.20 (Remote Device Management - RDM): Der Referenzstandard für die kaskadierende Abfrage und Entdeckung von Endpunkten über serielle Topologien hinweg. • OPC UA (IEC 62541): Der weltweit führende Standard für Industrie 4.0. Wichtig für GIOT-HAL sind das hierarchische Informationsmodell (Namespaces via String-Knoten) sowie das Daten-Trio bestehend aus Wert, Zeitstempel und Qualitätsstatus. • RFC 9562 (UUIDv4 / UUIDv5): Der offizielle IETF-Standard für die Strukturierung universell eindeutiger Identifikatoren, die im NVM des Geräts als Root-ID hinterlegt werden müssen.

1. Open-Source-Referenzimplementierungen und Code-Bibliotheken

Um die Entwicklung eines GIOT-HAL-Stacks zu beschleunigen und bestehende, praxiserprobte Routinen wiederzuverwenden, können die folgenden Open-Source-Projekte (unter permissiven Lizenzen wie Apache 2.0, MIT oder BSD) als Codebasis genutzt werden:

12.1 Für Layer 0 (Direct CPU Access) & OS-Kernel

• ARM CMSIS-Core: Offizielle Open-Source-Bibliothek von ARM. Bietet fertige Register-Abstraktionen, MPU-Konfigurationen und hardwarenahe Low-Layer-Zugriffe für alle Cortex-M Prozessoren. • Zephyr RTOS Kernel: Ein hochmodernes, modulares Echtzeitbetriebssystem für das IoT. Zephyr verfügt über ein mächtiges, deklaratives Treibermodell (Device Tree) und ein ausgereiftes Power-Management- und Sensor-Subsystem, das direkt als Blaupause für die Layer 1, 2 und 4 genutzt werden kann.

12.2 Für Layer 2 (Busse & IPC / Shared Memory)

• Embedded-HAL (Rust): Eine vollständig in Rust geschriebene, plattformunabhängige Hardware-Abstraktion. Sie nutzt das Konzept von "Traits", um universelle I2C-, SPI- und GPIO-Datenströme zu definieren, ohne an eine spezifische CPU gebunden zu sein. • OpenAMP (Asymmetric Multi-Processing): Die offizielle Referenzimplementierung des OpenAMP-Frameworks. Sie bietet fertigen C-Code für die sichere Interprozessorkommunikation (IPC) über Shared Memory mittels VirtIO und RPMsg und eignet sich perfekt für die asymmetrische Multi-Core-Schnittstelle in Layer 2.

12.3 Für Layer 3 & 4 (Konnektivität und Transformation)

• open62541 (OPC UA C-Stack): Eine vollständig in C (C99) geschriebene, extrem schlanke Open-Source-Implementierung von OPC UA. Sie ist frei von externen Abhängigkeiten, läuft auf Mikrocontrollern und stellt alle Routinen bereit, um Datenstrukturen inklusive Zeitstempel und Qualitätsstatus (Kapitel 3) regelkonform abzubilden. Damit ist dein Dokument vollständig strukturiert und bereit für LibreOffice. Wenn du möchtest, können wir als nächsten Schritt: • Einen vollständigen C-Code für den Pfad-Parser schreiben, der den String UUID::L2::L1::L0 zur Laufzeit auf dem Mikrocontroller zerlegt und validiert. • Oder das Dokument mit einer vorgefertigten Lizenz- und Urheberrechtsklausel abschließen, falls du die Spezifikation als Open-Standard veröffentlichen möchtest. Hier ist ein vollständiger Entwurf für ein Lizenz- und Urheberrechtskapitel (Kapitel 13), gefolgt von einer passenden Open-Source-Lizenzdatei. Beide Abschnitte sind im Markdown-Format gehalten, damit du sie direkt in deine Dokumentation übernehmen kannst. Für weltweite IoT-Standards hat sich die Apache-Lizenz 2.0 (Apache-2.0) als absolute Best Practice etabliert. Im Gegensatz zur MIT-Lizenz enthält sie eine explizite, gegenseitige Patentlizenzklausel. Da deine Spezifikation hardwarenahe Busse, Multiplexer und Algorithmen betrifft, schützt dich die Apache-2.0 davor, dass jemand den Standard nutzt, um im Nachgang Patentansprüche gegen dich oder die Community geltend zu machen.

1. Urheberrecht, Lizenzierung und Open-Standard-Governance

Um GIOT-HAL als weltweit akzeptierten, freien und barrierefreien IoT-Standard zu etablieren, wird diese Spezifikation sowie die zugehörigen Referenzimplementierungen unter einer permissiven Open-Source-Lizenz veröffentlicht.

13.1 Wahl der Lizenz (Apache License 2.0)

Die Spezifikation und der Code sind unter der Apache-Lizenz, Version 2.0 lizenziert. Diese Lizenz gewährt Anwendern und Halbleiterherstellern das dauerhafte, weltweite und nicht-exklusive Recht, diesen Standard kostenfrei zu nutzen, zu modifizieren und in kommerziellen Produkten (Closed-Source und Open-Source) ohne Lizenzgebühren (Royalties) zu implementieren.

13.2 Schutz vor Patenttrollen (Patent Grant)

Ein kritischer Aspekt bei Hardware-Schnittstellen ist das Risiko von Patentverletzungen. Die Apache-2.0-Lizenz beinhaltet eine explizite, gegenseitige Patentlizenzierung: • Jeder Contributor, der Code oder Spezifikationsteile beisteuert, gewährt allen Nutzern automatisch eine Patentlizenz für seine in diesem Standard enthaltenen Patente. • Sollte ein Unternehmen ein Patentverfahren gegen einen GIOT-HAL-Nutzer einleiten mit der Behauptung, der Standard verletze ein Patent, erlöschen die Patentrechte dieses Unternehmens für GIOT-HAL mit sofortiger Wirkung.